

03b — Asynchronous Composition

Interleaved Execution

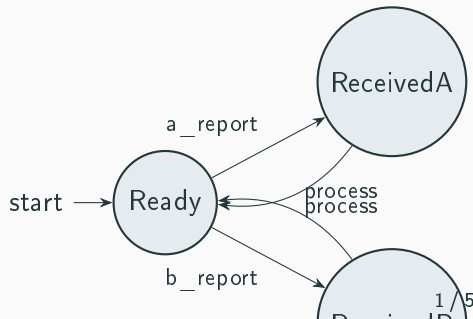
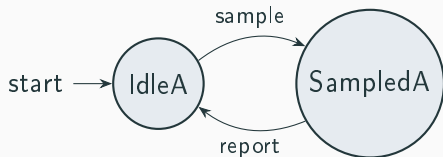
Mariano Cerrutti

Mununu Tutorial

System Overview: Two Sensors + Monitor

Three independent components communicating via shared labels:

- **SensorA** — cycles between `IdleA` and `SampledA`
- **SensorB** — cycles between `IdleB` and `SampledB`
- **Monitor** — receives reports: `Ready`, `ReceivedA`, `ReceivedB`



Automata Definitions

SensorA and **SensorB** — simple two-state cycles:

```
1      automaton SensorA {  
2          // Sensor controls sampling; reporting is environment-triggered  
3          controllable {  
4              label sensor_a_sample;  
5          }  
6  
7          states {  
8              state IdleA initial;  
9              state SampledA;  
10         }  
11  
12         transitions {  
13             transition IdleA -> SampledA on label sensor_a_sample;  
14             transition SampledA -> IdleA on label sensor_a_report;  
15         }  
16     }
```

Asynchronous Composition

```
1      asynchronous sensor_network {  
2          members [SensorA, SensorB, Monitor];  
3      }  
4  }
```

Key difference from synchronous:

- Any component can step *independently* — no lock-step.
- No `skip` labels needed.
- Shared labels (e.g. `sensor_a_report`) still synchronize the components that use them.
- Non-shared labels execute as *interleaved* steps.

State space: $2 \times 2 \times 3 = 12$ composed states.

Properties over the Composition

```
1      // Safety: the sensor network always stays in valid states
2      formula safety {
3          over sensor_network;
4          body = nu X. ([] X);
5      }
6
7      // No deadlocks in the composed system
8      formula no_deadlock {
9          over sensor_network;
10         body = nu X. ((<> true) && ([] X));
11     }
12
13     // Properties over individual automata
14     formula sensor_a_can_sample {
15         over SensorA;
16         body = mu X. (SampledA || <> X);
17     }
```

Synchronous vs. Asynchronous Comparison

Aspect	Synchronous	Asynchronous
Execution model	Lock-step (all move together)	Interleaved (any can move)
Skip labels	Required for “no-op” steps	Not needed
Shared labels	All participate on every step	Only involved parties sync
State space	Product of states	Product of states
Typical use	Tightly coupled components	Independent components

Rule of thumb: use [synchronous](#) when components must coordinate on every step; use [asynchronous](#) when components act independently and synchronize only on shared events.